

Practical-4

Matrix Chain Multiplication Using Dynamic Programming

Theory :-

Matrix Chain Multiplication (MCM) is a Dynamic Programming problem used to find the minimum number of scalar multiplications required to multiply a chain of matrices.

The order of the matrices cannot be changed, but the placement of parentheses can be changed. Different parenthesizations can require different numbers of multiplications.

For matrices:

$$A_1 A_2 A_3 \cdots A_n$$

the objective is to find the parenthesization that gives the minimum multiplication cost.

Code :-

```
//Chain Matrix Multiplication Using Dynamic Programming
#include <iostream>
#include <climits>
using namespace std;

int chainMatrix(int p[],int n){
    int dp[n][n];

    // We filling the diagonal part with zero
    for (int i = 1 ; i < n ; i++){
        dp[i][i] = 0;
    }

    for (int len = 2 ; len < n ; len++){
        for (int i = 1 ; i < n-len+1 ; i++){
            int j = i + len-1;
            dp[i][j] = INT_MAX;

            for (int k = i ; k < j ; k++){
```

```

        int cost = dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j];

        if(cost < dp[i][j]){
            dp[i][j] = cost;
        }
    }
}

return dp[1][n-1];
}

int main(){
    int n ;
    cout << "Enter number of Matrix : " ;
    cin >> n ;

    int p[n+1];
    cout << "Enter the dimensions of the matrix : ";
    for(int i = 0 ; i <= n ; i++){
        cin >> p[i];
    }

    int result = chainMatrix(p,n+1);

    cout << "Minimum Multiplication cost is : "<<result;
}

```

Output :-

Problems Output Debug Console **Terminal** Ports

```

● PS C:\Users\shiva\Desktop\UNI Coding\DAA> cd "c:\Users\shiva\Desktop\UNI Coding\DAA"
1 } ; if ($?) { .\Practical_6_Chain_Matrix_Mul }
Enter number of Matrix : 3
Enter the dimensions of the matrix : 10
20
30
40
Minimum Multiplication cost is : 18000
○ PS C:\Users\shiva\Desktop\UNI Coding\DAA>

```

Analysis Of Algorithm :- Chain Matrix Multiplication

- **Input:** Number of matrices and their dimensions.
- **Output:** Minimum number of scalar multiplications.
- **Technique:** Dynamic Programming.
- **Time Complexity:** $O(n^3)$
- **Space Complexity:** $O(n^2)$

Where n is the number of matrices.